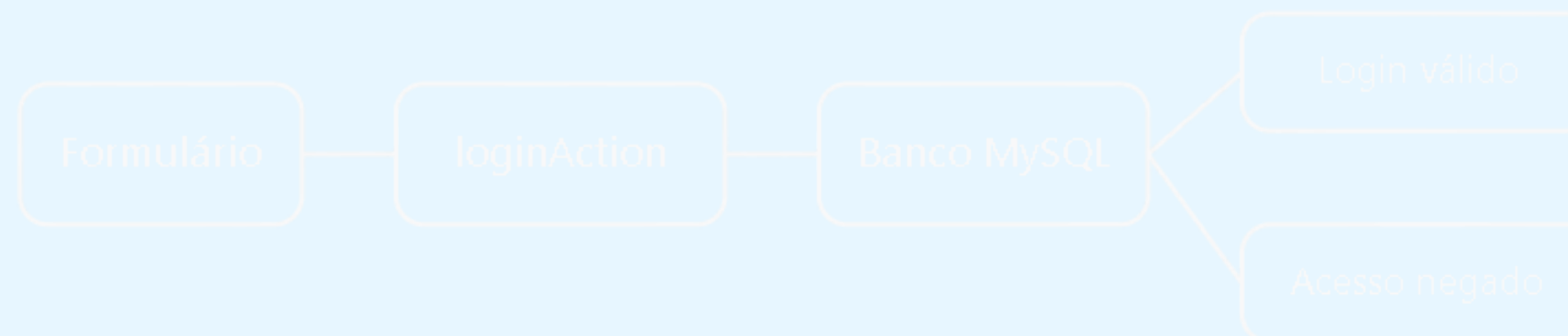


Feito por: Pedro Henrique Fukassawa da Silva



Sistema CRUD com Login e Controle de Acesso

Estrutura do banco de dados

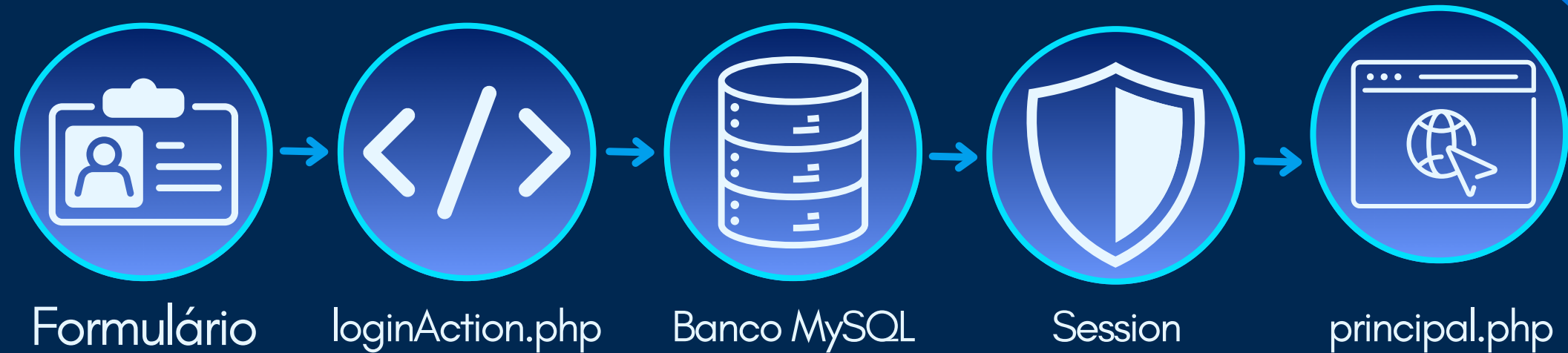
Usuário (login)

Campo	Função
id	Identificador
nome	Usuário
senha	Senha



Armazena as credenciais utilizadas para validar o acesso ao sistema

Como funciona o login?



O formulário envia os dados para loginAction.php, o PHP consulta a tabela usuario, compara a senha e, se estiver correta, cria uma SESSION que libera o acesso à página principal.



Destaque Técnico

- Validação feita via "SELECT";
- "Session" impede o acesso direto pelo URL;
- Conexão, cabeçalho e rodapé reutilizados com require_once.

Fluxo completo do login



Usuário

Senha

Entrar

- 1 Usuário digita nome e senha.
- 2 Formulário envia para loginAction.php.
- 3 O PHP consulta o banco de dados.
- 4 Senha é comparada.
- 5 O acesso é liberado ou negado.

Como a autenticação ocorre



O que o PHP faz

- Recebe txtNome.
- Recebe txtSenha.
- Procura o usuário.
- Compara a senha.
- Cria uma sessão quando tudo está correto

Controle de acesso

Acesso verificado via
“verificarAcesso.php”

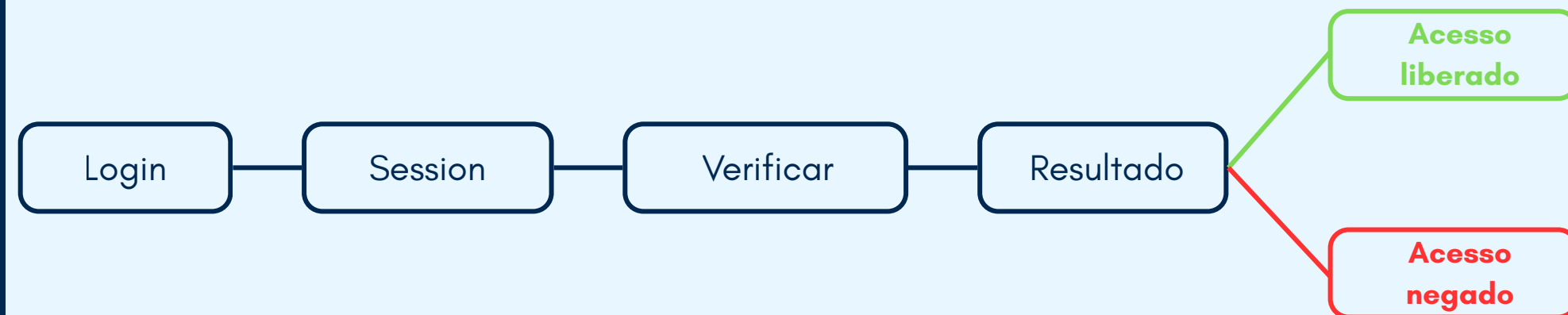
Acesso NEGADO!

É necessária a Realização do **Login válido!**

🔧 O que o “verificarAcesso.php” faz

- Verifica se existe uma sessão.
- Confere se `$_SESSION['logado']` existe.
- Se não existir:
 - Usa `header()`;
 - `header("location:acessoNegado.php");`
 - Redireciona para `acessoNegado.php`.

O que o Session faz



🔧 Objetivo da Session

- mantém o usuário autenticado;
- impede acesso direto pela URL;
- exige novo login após o logout.

É utilizado os seguintes códigos para a criação e utilização da sessão

- `session_start();`
- `$_SESSION['logado']`

Quando o login é válido, a sessão é criada.

O **CRUD** do sistema



Create

Cadastrar amigo



Read

Listar amigos



Update

Edita amigo cadastrado



Delete

Exclui amigo cadastrado

Cada operação corresponde a
comandos SQL diferentes.

Create (Cadastrar)

⚙️ Funcionamento do Create

1 Preenche o formulário.

2 Envia via **POST**.

3 O PHP recebe os dados enviados via **POST**.

4 Executa **INSERT**.

SQL utilizado:

INSERT INTO amigo (...)
VALUES (...);

5 Salva no banco.

Read (Listar)

⚙️ Funcionamento do Read

1 O sistema executa:
SQL executado -

```
SELECT * FROM amigo;
```

2 Depois utiliza:

- **foreach** - Percorre cada amigo retornado pelo banco.
- **echo** - Imprime cada linha da tabela na página.

Para montar dinamicamente a tabela.

Update (editar)

⚙️ Processo do Update

1 Seleciona um amigo.

2 Carrega os dados.

O formulário é preenchido automaticamente com as informações já existentes

3 Altera as informações.

4 Executa o **Update**:

```
UPDATE amigo
SET nome = (...)
WHERE idamigo = ?;
```

Importância

Evita criar cadastros duplicados.

Delete (Excluir)

⚙️ Funcionamento do Delete

- Ao escolher excluir, o seguinte código é executado:

```
DELETE FROM amigo
WHERE idamigo = ?;
```

O registro é removido do banco.

Seleciona

Confirma

Registro removido



CANCELAR EXCLUSÃO

EXLUIR - ID: 1

Nome

Gabriela

Apelido

Gabi

Email

gabi@gmail.com



Confirmar Exclusão.

Importação de arquivos (require_once)

Em vez de repetir o mesmo código em vários arquivos, é usado o require_once.

conexaoBD.php

Reúne todas as informações de conexão (host, usuário, senha e banco). Assim, qualquer alteração é feita em um único arquivo.

```
require_once 'conexaoBD.php';
```

cabecalho.php

Guarda o início do HTML, incluindo <head>, links do W3.CSS e abertura do <body>, mantendo todas as páginas com o mesmo padrão visual.

```
<?php require_once  
( 'cabecalho.php' ); ?>
```

rodape.php

Fecha a estrutura do HTML com </body> e </html>, evitando repetir o mesmo código em todos os arquivos.

```
<?php require_once  
( 'rodape.php' ); ?>
```

Conclusão

O projeto demonstra um fluxo completo do desenvolvimento de um site de login e lista com autenticação, proteção por sessão e operações de banco de dados organizadas em uma CRUD (Create, Read, Update e Delete).